Topic 4: Python Programming

Learning Outcomes:

(a) i . Define types of programming languages, programming paradigms and language translators

Introduction to Programming Language

- A programming language is a way to **communicate with a computer.**
- It allows us to **write instructions** that tell the computer what to do.
- These instructions **follow a specific syntax** (structure) and rules.

Introduction to Programming Language

- Programming languages are used to **build** software, games, websites, and more.
- **Examples** of programming languages include Python, Java, C++, and JavaScript.
- Each language has its own strengths and is suited for different tasks.

DEFINITION - Programming Language

What is **Programming Language**?

- A ***programming language*** is a formal set of rules used to write instructions that a computer can understand and execute.

Source: Britannica, 2024

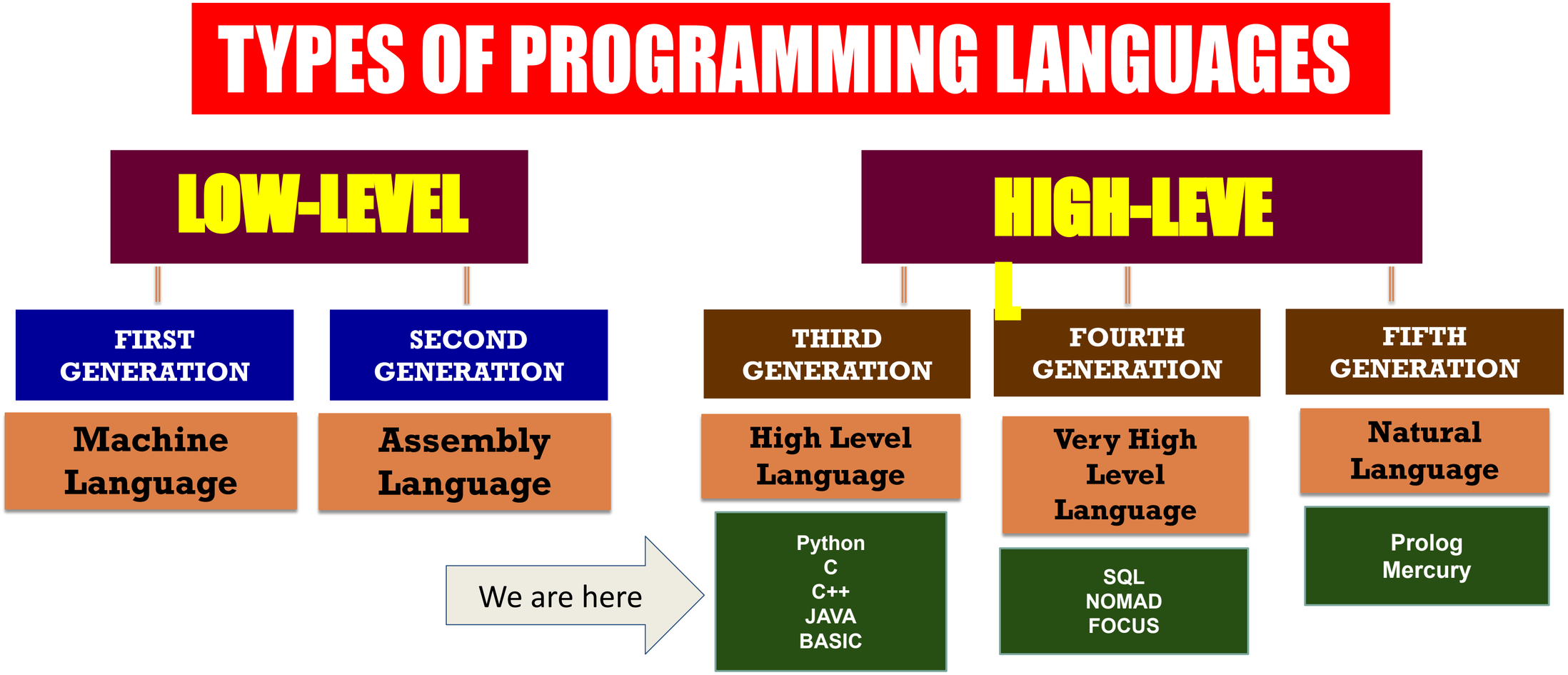
COMPUTER PROGRAMMING

LECTURE NOTES

CS125

PROGRAMMING LANGUAGE

Types of Programming Language:



Types of Programming Language:

Low-Level Language

- A **low-level language** is a programming language that is *machine-dependent*.
  - A *machine-dependent language* runs on only one particular type of computer.
  - These programs are not easily portable to other types of computers.

Low-Level Language: Machine Language

- ***Machine language*** is defined as a language that consists of strings of binary digits (1 and 0) to represent instructions to computer.
- It is the natural language of a computer.

To calculate *wages = rates * hours* in machine language:

```
100100 010001  // Load (input rates)
100110 010010  // Multiply with hours
100010 010011  // Store (result of wages)
```

Low-Level Language: Machine Language

Advantages

1.Ready for immediate execution.

2.Instructions are readily understood by the computer.

Disadvantages

1.Machine-dependent.

2.Programs tend to be lengthy.

3.Coding in machine language is tedious and time-consuming.

Low-Level Language: Assembly Language

- ***Assembly language*** is the second generation of programming languages consists of *English-like abbreviations*.
- Programmer writes instructions using symbolic instruction codes. Symbolic instruction codes are meaningful abbreviations.

Example: x86, arm a32

Example:

ADD for *addition*
CMP for *compare*
LOAD for *load*
MULT for *multiply*

Low-Level Language: Assembly Language

Advantages

1.Instructions are easier to learn compared to machine language.

2.Easy to understand and use. Assembly language use mnemonics instead of using binary code.

Disadvantages

1. Machine-dependent.
2. Long and tedious to write.

Low-Level Language:

Machine Language

```
10110000 01100001
10110010 01000010
00000001
11100010
```

Assembly Language

```
MOV AL, 61h
MOV DL, 42h
ADD AL, 1
LOOP
```

Types of Programming Language:

High-Level Language

- A **high-level language** is a programming language that is closer to *natural language* and easier to work with than a low-level language.
  - A high-level language is a programming language that is *machine-independent*.
  - A *machine-independent* language can run on many different types of computers and operating systems.

High-Level Language

The equation : *wages = rate x hours* can be written in Python as:

```
wages = rate * hours
```

Examples of high-level language:
Python, Java, C, C++, BASIC, Pascal, FORTRAN (for Scientific), COBOL (for Business)

High-Level Language:

Advantages

1. The English-like instructions are easier to learn compare to low-level language.
2. The instructions can run on many different types of computers

Disadvantages

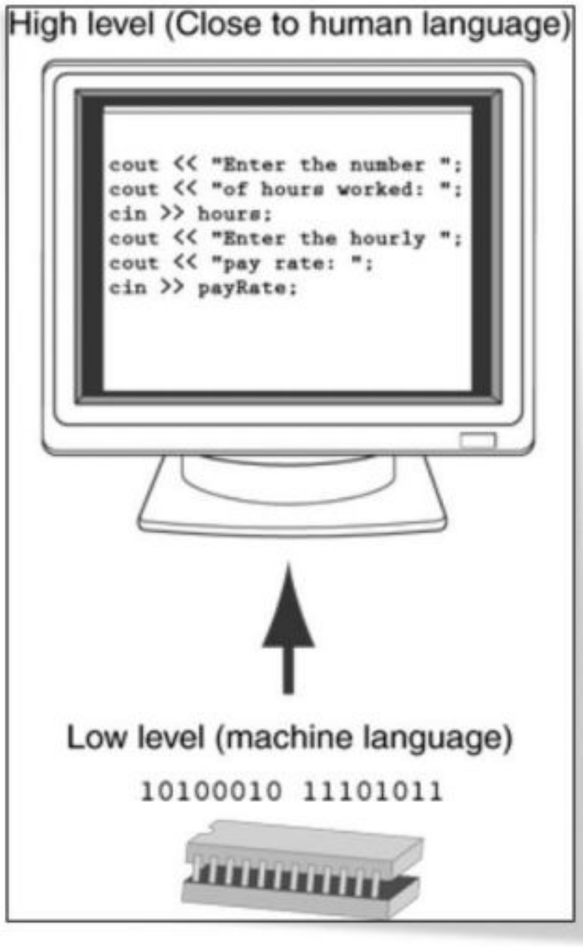
1. Not efficient as low level languages.
2. Program generally run slower. (Need to be translated / compile)

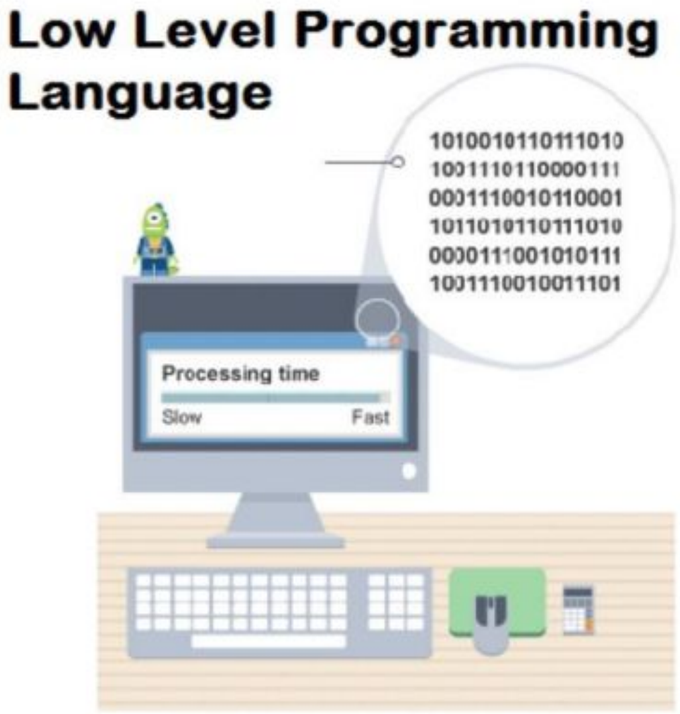
COMPUTER PROGRAMMING

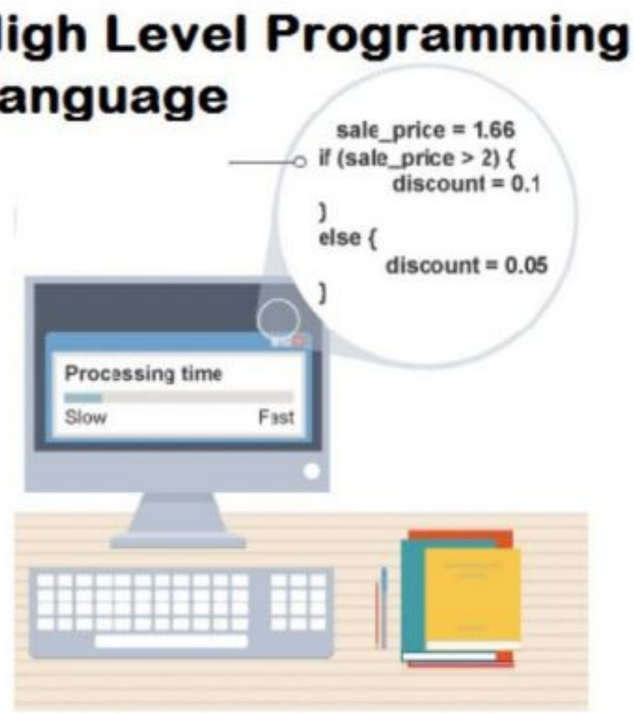
LECTURE NOTES

CS125

PROGRAMMING LANGUAGE







LECTURE NOTES

Low-Level Language	High-Level Language
Consists of binary digits or English-like abbreviations to form instructions.	Consists of English-like words to form instructions.
Machine Language does not need translator.	Need translator to convert to low-level language.
Machine dependant.	Machine independent.
Difficult to learn, modify and far from human language.	Easy to learn, modify and close to human language.

LECTURE NOTES

QUICK REVIEW

```
# Get the first number from the user
num1 = int(input("Enter the first number: "))

# Get the second number from the user
num2 = int(input("Enter the second number: "))

# Calculate the sum
sum = num1 + num2

# Print the sum
print("The sum of", num1, "and", num2, "is", sum)
```

1. State the type of programming language shown above.
2. Give one benefit of writing code in your answer above.

Introduction to Programming Paradigm

- 🧠 **A programming paradigm** is a way of thinking about and writing code — like a coding "style" or approach.
- 💡 Different paradigms (like procedural, object-oriented, and logical) help solve problems in different ways.
- 🛠️ Programming languages often support one or more paradigms to give developers flexibility when building software.

DEFINITION - Programming Paradigm

What is **Programming Paradigm**?

- A ***programming paradigm*** is a fundamental style or approach to programming that guides how code is written, structured, and executed.

Source: Britannica, 2024

Types of programming paradigm:

PROCEDURAL	OBJECT-ORIENTED	LOGIC

Procedural Programming vs Object Oriented Programming

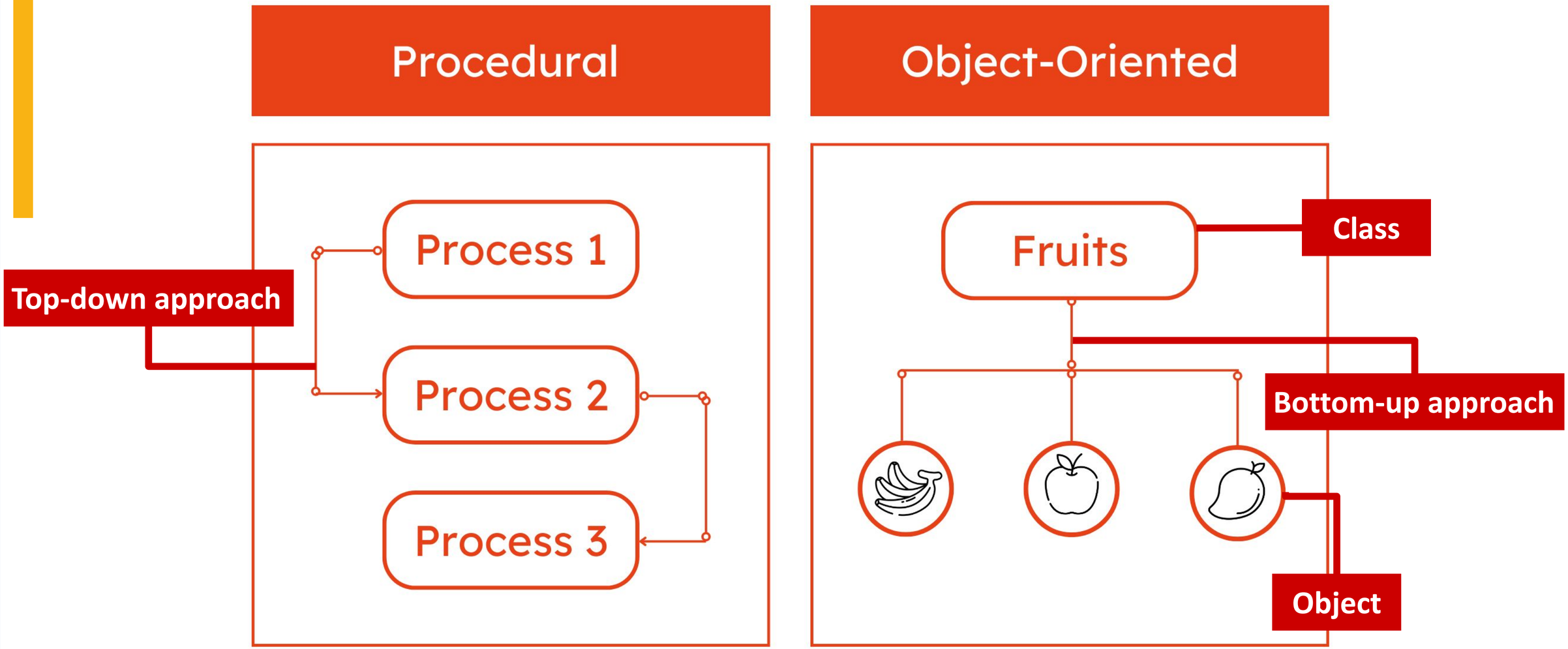
Procedural Programming	Object-Oriented Programming
In procedural programming, the program is divided into small parts called ***functions***.	In object-oriented programming, the program is divided into small parts called ***objects***.
Procedural programming follows a ***top-down approach***.	Object-oriented programming follows a ***bottom-up approach***.
Examples: C, FORTRAN, Pascal, Basic, Python etc.	**Examples:** C++, Java, Python, C#, etc.

COMPUTER PROGRAMMING

LECTURE NOTES

CS125

PROGRAMMING PARADIGM



Logic Programming

Logic Programming
In logic programming, the program is expressed in terms of ***facts***, ***rules***, and ***queries***.
Logic programming follows a ***declarative approach*** (focuses on ***what*** to solve rather than ***how*** to solve).
Examples: PROLOG, DATALOG, ASP, etc.

COMPUTER PROGRAMMING

LECTURE NOTES

CS125

PROGRAMMING PARADIGM

Procedural Programming / Object Oriented Programming



Logic Programming



Source: https://dev.to/stm-akikaze1119/understanding-the-difference-between-imperative-and-declarative-programming-1j3m

Introduction to Language Translator

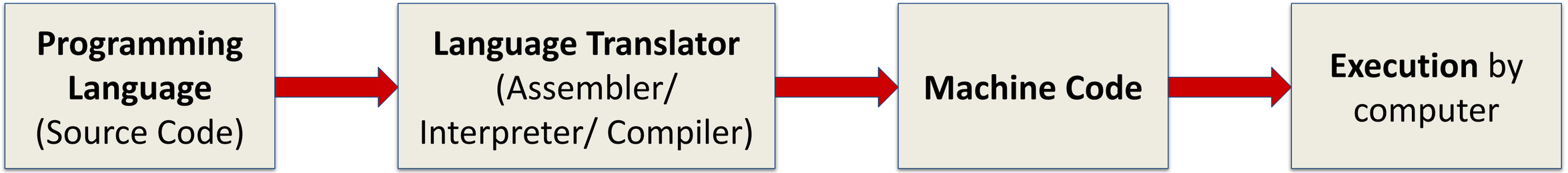
- 🧾 A language translator is a program that converts code written in one programming language into another (usually machine language).
- 💻 It helps computers understand and execute code written by humans in high-level languages like Python, Java, or C++.
- 🛠️ There are three main types: Assembler, Interpreter, and Compiler, each translating in different ways.

DEFINITION - Language translator

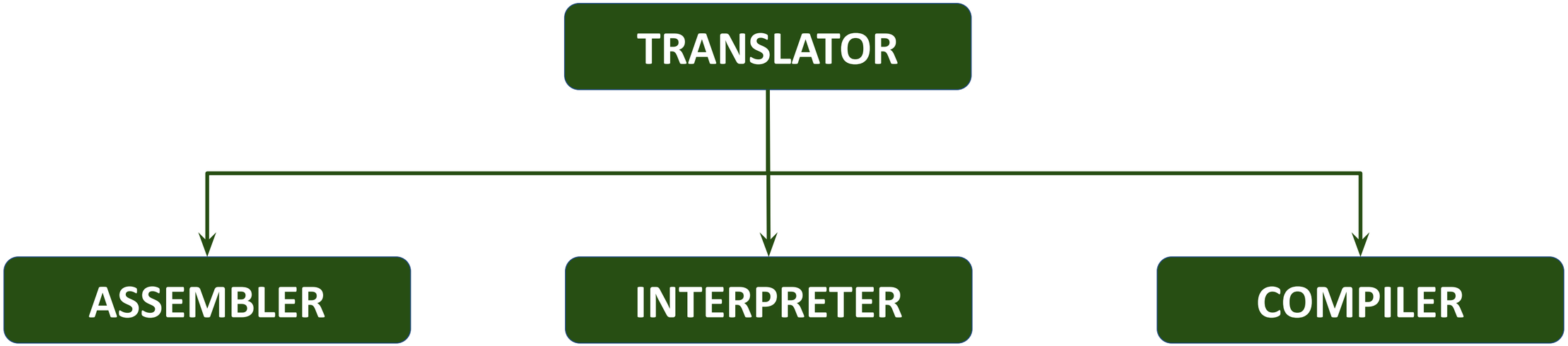
What is **Language Translator**?

- A ***language translator*** is a program that converts instructions written in one programming language into machine code

Source: Computers for Librarians (Third Edition), 2003

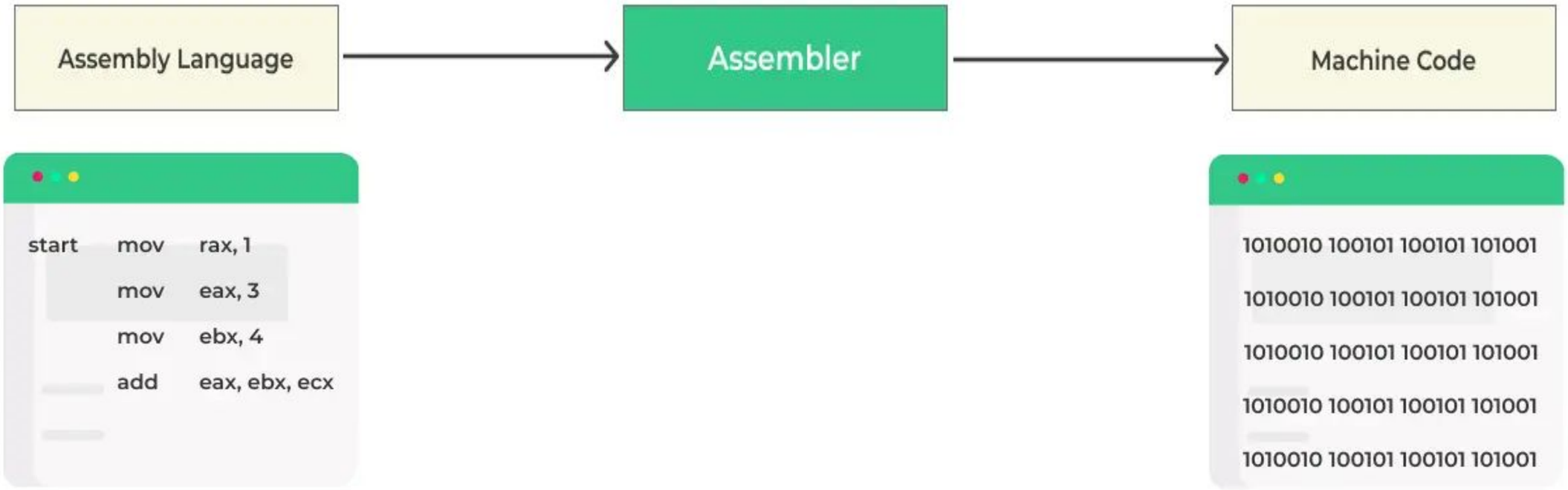


Types of Language Translator:



1. Assembler

A program that translates **assembly language into machine language.**



1. Assembler

Examples:-

- **NASM** (Netwide Assembler)
- **MASM** (Microsoft Macro Assembler)
- **TASM** (Turbo Assembler)
- **FASM** (Flat Assembler)

2. Interpreter

A program that translates translate the program's high level instructions **line by line** into machine language instructions as the program is running.



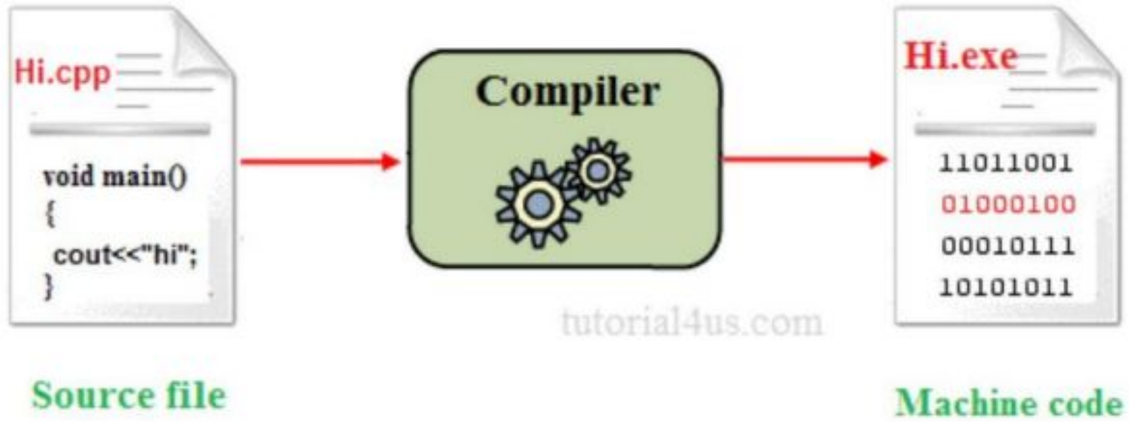
2. Interpreter

Examples interpreted programming language:

- **Python** programming language
- **Ruby** programming language
- **Javascript** programming language

3. Compiler

A program that translate **all of a program's high-level instructions** into machine language instructions before running the program



3. Compiler

Examples

- **Java** programming language
- **C++** programming language
- **C** programming language
- **Fortran** programming language
- **ADA** programming language
- **Pascal** programming language
- **Kotlin** programming language

Topic 4: Python Programming

Learning Outcomes:

4.1 ii. Identify available platforms for writing Python programs.

Requirements to Use Python

The standard implementation of Python is written in portable ANSI C, and it compiles and runs on virtually every major platform currently in use. As a partial list, Python is available on:

- Linux and Unix systems
- Microsoft Windows
- Mac OS
- Cloud platform

Types of Python Platforms

There are two main types of Python platforms: **Online-Based** and **Local-Based**.

- **Online-Based platforms** allow you to write and run Python code directly in a web browser, with executions happening remotely on cloud servers.

- **Local-Based platforms** require installation on your local machine, where Python code is executed using your device's own resources.

Online-Based Python Platforms

Your code runs on **remote servers (the cloud)**. You just need a browser and an internet connection. All processing and file storage typically happen online.

Advantages:

- **No setup** needed — just open a link and start coding
- **Accessible anywhere** with an internet connection
- Easy to **share and collaborate** with others in real-time
- Often includes **free computing resources** like GPUs (e.g., Google Colab)

Online-Based Python Platforms

Limitations:

- Requires **stable internet**
- May have **performance limits** (especially with free versions)
- Not ideal for **complex software or offline work**

LECTURE NOTES

Online-Based Python Platforms

Google Colab

Google Colab is a **free browser-based Python notebook** provided by Google. It lets you write and execute Python code on cloud servers



Allows you to write code in **cells**, with outputs shown below each cell. Supports uploading datasets and saving notebooks to **Google Drive**

Local-Based Python Platforms

You download and install the software (e.g., PyCharm, Jupyter Notebook). It uses your computer's resources to run the Python interpreter and handle files.

Advantages:

- **No internet required** after installation
- Usually **faster and more powerful**, especially for large projects
- Fully **customizable and extendable** with plugins or libraries
- Better control over **local files, packages, and environment**

Local-Based Python Platforms

Limitations:

- Requires installation and setup
- Takes up disk space
- Can be quite technical for beginners to install

Local-Based Python Platforms

PyCharm

PyCharm is a **professional-grade IDE (Integrated Development Environment)** for Python development, created by JetBrains. It supports full software development workflows.

It is a powerful code editor with intelligent suggestions and refactoring. It also have integrated debugger, version control, and terminal.



Jupyter Notebook

Jupyter Notebook is an **interactive web-based tool** for coding in Python. It lets you write code, add notes, display charts, and run data analysis.

Even though it runs through the browser, it stills run locally on your computer using your computer resources

It split your code into cells for easier execution and experimentation. It also can combine code + text (Markdown) + output (charts, tables).



Platforms Comparison

Platform	Type	Ease of Use	Key Features
Google Colab	Online	Very Easy	No setup required, free access to resources such as CPU and RAM
PyCharm	Local	Moderate (Require setup)	Full-featured IDE with project management and AI coding assistant
Jupyter Notebook	Local	Moderate (Require setup)	Interactive cells with live code and outputs

Use Cases and Suggested Platforms

Running Python Without Installation

- Suitable for trying out Python without setting up a development environment.
- Runs entirely in a web browser with no need for local installation.
- Convenient for quick testing and experimentation.

Suggested Platform: **Google Colab**

Use Cases and Suggested Platforms

Managing Larger Projects with Multiple Files

- Ideal for structured projects that involve multiple scripts, folders, or modules.
- Supports features like project navigation, debugging tools, and version control.
- Suitable for more advanced or long-term development work.

Suggested Platform: **PyCharm**

Use Cases and Suggested Platforms

Interactive Coding with Instant Output

- Supports writing and executing code in smaller, organized cells.
- Provides immediate visual output for each section of code.
- Well-suited for data visualization, exploration, and explanation.

Suggested Platform: **Jupyter Notebook**

Topic 4: Python Programming

Learning Outcomes:

4.3 Identify components of a Python program:

- identifiers
- variables
- reserved words/keywords
- data types
- output statements
- input statements
- import statements
- comments
- indentation

COMPUTER PROGRAMMING

LECTURE NOTES

CS125

Table of Contents

Component	Description
A. Naming Elements	Basic rules for using identifiers, variables, and reserved words/keywords in a Python program.
B. Data Types	Basic data types: integer, float, Boolean, string, and list. Includes basic string operations using `len()` and concatenation.
C. Basic Input Output	Using `print()` to display output, `type()` to identify data types, `input()` to receive input, and casting to convert data types.
D. **Using Python Module**	Using the `import` statement to access functions and constants from the `math` module.
E. Program Formatting	Using indentation to structure Python code and comments to document the program.

COMPUTER PROGRAMMING

LECTURE NOTES

CS125

Table of Contents

A. NAMING ELEMENTS
   1. identifiers
   2. variables
   3. reserved words / keywords

COMPUTER PROGRAMMING

LECTURE NOTES

Definition of Identifier:-
A name used to identify components in a Python program, such as variables, function name, class name.

LECTURE NOTES

Rules for Naming Identifiers:

1. **Can contain letters (a-z, A-Z), digits (0-9), and underscores (_)**
   - ✅ Valid: `my_var, speed1, Calculate_Area`
   - ❌ Invalid: `my-var` (hyphens are not allowed)

2. **Can use underscores but not special characters (@, $, %)**
   - ✅ Valid: `_my_variable, count_1`
   - ❌ Invalid: `my$var, #number`

3. **Cannot start with a number**
   - ✅ Valid: `age1 = 25, _value = 50`
   - ❌ Invalid: `1variable = 10` (SyntaxError)

Rules for Naming Identifiers:

4. Case-sensitive

- `Variable` and `variable` are different identifiers.

Example:

```
age = 25
Age = 30
print(age)   # Output: 25
print(Age)   # Output: 30
```

5. Cannot be a reserved word (keyword)

- ❌ Invalid: `class`, `def`, `if`, `while`, `return`

LECTURE NOTES

Meaningful Identifiers:

- Use meaningful names that describe the purpose of the data.

A meaningful identifier tells us what the variable or function represents.

✓ *Recommended*	✗ *Not Recommended*
`student_name` → variable	`x` → unclear
`total_marks` → variable	`tm` → unclear abbreviation
`calculate_average` → function	`Func1` → does not describe the task

Meaningful Variable Names:

- Choose a name that describes the value stored in the variable

✓ Recommended		✗ Not Recommended	
`student_name`	= “Aina”	`x`	= “Aina”
`total_marks`	= 85	`y`	= 85
`age`	= 18	`a`	= 18

Can you understand what the data represents just by reading the variable name?

Meaningful Function Names:

- A function name should describe the task performed by the function.

✓ Recommended	✗ Not Recommended
`calculate_average`	`func1`
`display_result`	`do_it`
`get_student_name`	`abc`

snake_case Naming Convention:

The recommended naming convention for **variables and functions in Python** is **snake_case**.

In snake_case:

- Use **lowercase letters**.
- Separate multiple words using an **underscore (_)**.
- Use **meaningful words** that describe the purpose of the variable or function.

snake_case Naming Convention:

Words are written in lowercase and joined using an underscore (_).

```
snake_case = lowercase words + underscore (_)
```

- snake_case makes identifiers easier to read, understand, and maintain.

Difficult to read	***Easier to read***
`totalstudentmarks`	`total_student_marks`

LECTURE NOTES

snake_case Naming Convention:

Variable examples	*Function name examples*

`student_name = "Aina"`
`total_marks = 85`
`average_score = 75.5`
	`calculate_average`
	`display_result`
	`get_student_name`

Comparison of Naming Conventions

Naming Convention	Example	Naming Pattern	Common Use
snake_case	`student_name`	Lowercase words separated by _	Python variables & functions
camelCase	`studentName`	First word lowercase; following words start with uppercase	Common in Java
PascalCase	`StudentName`	Every word starts with uppercase	Common for class names



Review Activity : Identifier

Try this !  **Exercise 1**

Which of the following is a valid Python identifier?

A. `2name`

B. `name_2`

C. `For`

D. `class-name`

COMPUTER PROGRAMMING

LECTURE NOTES

CS125

2. VARIABLE

Definition of Variable:-
A named storage location used to store a value in a program.

- A variable has a name and stores a value.
- In Python, we do not need to specify the data type when creating a variable.
- Python automatically determines the data type based on the value assigned.

LECTURE NOTES

Examples	*Explanation*
`student_name = "Aina"`	`student_name = "Aina"` ↑ variable name ↑ value
`age = 18`	`age` → variable name `=` → assignment operator `18` → value

Variable Assignment / Initialization
The assignment operator (=) assigns a value to a variable.

Example	`student_name = "Aina"`
	`age = 18`
	`height = 1.65`

Variable Assignment / Initialization
A variable's value can also be changed.

Example	`score = 70` `score = 85`

* The current value of score is **85**.

COMPUTER PROGRAMMING

LECTURE NOTES

CS125

2. VARIABLE

Feature	Identifiers	Variables
Definition	Name of functions, classes, and variables	Name referring to stored data
Purpose	Used to identify elements in code	Stores values that can change during execution
Example	`my_function, Student, count`	`x = 10, name = "John"`
Keywords	Cannot use reserved words	Variable names should not be reserved words



Review Activity : Variable

Try this !  **Exercise 1**

Which of the following is **NOT** a valid variable name in Python?

a) `my_var`
b) `total1`
c) `break`
d) `dataSet`



Review Activity : Variable

Try this ! **Exercise 2**

What is the **purpose** of a variable in Python?

a) To reserve memory for loops
b) To perform calculations
c) To store data values
d) To create comments



Review Activity : Variable

Try this !  **Exercise 3**

Is `Total_Score` and `total_score` the same variable in Python?

COMPUTER PROGRAMMING

LECTURE NOTES

Definition of Reserved Word:-
A word that has a special meaning in Python and cannot be used as an identifier.

- Also known as **keyword**.
- They are part of the Python language.

Characteristics of Reserved Words:

- **Cannot be used as identifiers** – You cannot name your variables, functions, or classes using reserved words.
- **Predefined meanings** – Each reserved word has a specific role in Python syntax.
- **Case-sensitive** – Python keywords must be written exactly as they are defined (e.g., True is different from true).
- **Fixed list** – The number of reserved words may change with new Python versions.

A list of some common reserved words in Python (cont.) :

Reserved Word	Meaning
`if`	Used for conditional statements
`else`	Defines an alternative condition
`elif`	Used in multiple condition statements
`for`	Starts a loop that iterates over a sequence
`while`	Starts a loop that runs as long as a condition is true
`break`	Exits the current loop
`continue`	Skips the current iteration of a loop and moves to the next
`def`	Defines a function

A list of some common reserved words in Python :

Reserved Word	Meaning
`return`	Returns a value from a function
`class`	Defines a class
`import`	Imports modules into a program
`try`	Starts a try-except block for handling exceptions
`except`	Catches exceptions in a try-except block
`True`	Boolean value for true
`False`	Boolean value for false
`None`	Represents a null value

Invalid	*Valid*
`if = 10`	`student_mark = 10`

Python keywords are case-sensitive.

For example:

```
True
```

`True` is a **Python keyword** representing a Boolean value.



Review Activity : Reserved Word

Try this ! **Exercise 1**

Which of these is a Python reserved word (keyword)?

a) `name`
b) `value`
c) `def`
d) `var`



Review Activity : Reserved Word

Try this !  **Exercise 2**

What will happen if you use a reserved word like if as a variable name?

a) It will run normally
b) It will be ignored by Python
c) It will cause an error
d) It will change the value of if

Table of Contents

B. DATA TYPES

4.1 Integer (`int`)

4.2 Float (`float`)

4.3 Boolean (`bool`)

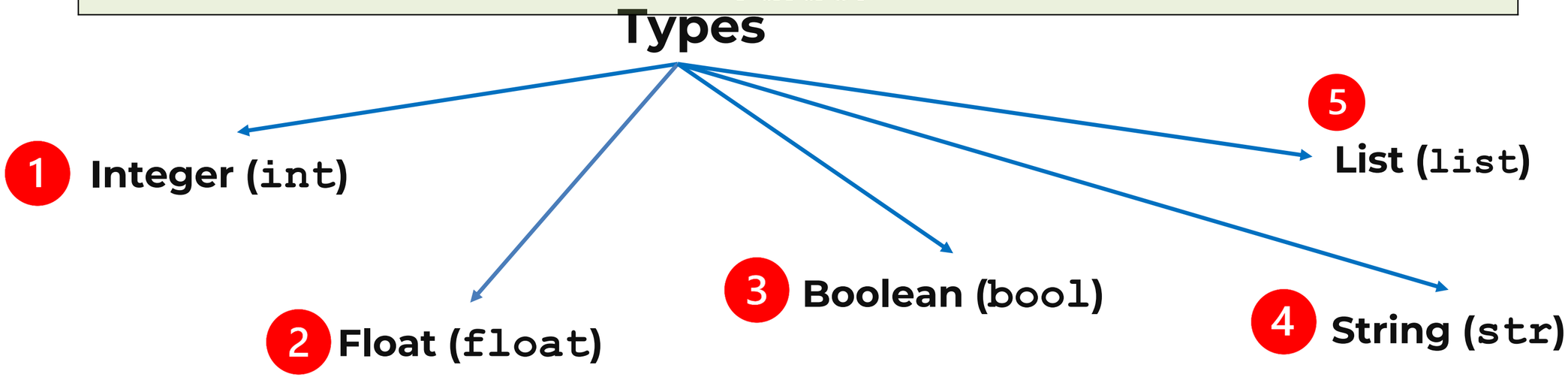
4.4 String (`str`)

4.5 List (`list`)

LECTURE NOTES

Definition of Data Type:-

The data type of a value specifies how the value is stored in the computer and what operations can be performed on the value.



LECTURE NOTES

1 Integer (`int`)

- ***A whole number without a decimal point.***
- Represents whole numbers (positive, negative, or zero)

Example	*Integer values*
`age = 18`	`18`
`score = 100`	`0`
`temperature = -5`	`-25`

2 Float (`float`)

- ***A number that contains a fractional part***.
- Represents real numbers (numbers with a decimal point).
- Python does not have a double data type like Java or C++, as float in Python already provides double-precision floating points.

Example	*float values*
`price = 12.50`	`12.50`
`average = 75.5`	`0.5`
`temperature = -2.5`	`-3.25`

3 Boolean (`bool`) ***`True` and `False` are Python keywords and are case-sensitive.***

- Represents one of two logical values: True or False (case-sensitive).
- Commonly used in conditions and comparisons.
- Booleans are essentially integers (True = 1, False = 0) and can be used in arithmetic operations.
- uses **capitalized first letters**

Example	*Boolean values*
`is_student = True`	`True`
`is_absent = False`	`False`

- *`True` and `False` are Python keywords and are case-sensitive*

4 String (`str`)

- ***A sequence of characters enclosed in a pair of single or double quotation marks***.
- Represents a sequence of characters (text data).
- Strings can be enclosed in single ('), double (") or triple (''' """) quotes
- Strings are immutable (cannot be changed after creation).

Example Single quotation marks	*Example Double quotation marks*
`name = 'Aina'` `course = 'CS125'`	`name = "Aina"` `course = "CS125"`

4 String (`str`)

What Can a String Contain?

- A string can contain letters, numbers, spaces, and symbols.

`"Hello"`	Letters
`"Room 12"`	Letters, numbers and space
`"Aina Ahmad"`	Letters and space
`"RM25.50!"`	Letters, numbers and symbols

- *Quotation marks identify the value as a string.*

4 String (`str`)

1. Basic String Operation: `len()` Function

- Purpose : *to find the number of characters in a string.*
- Spaces and symbols are also counted as characters.

Syntax	*Example*
`len(variable_name)`	`name = "Aina"` `len(name)`

** Number of characters: 4*

4 String (`str`)

2. Basic String Operation: `Concatenation`

- Definition : *The process of joining two or more strings together.*
- Operator (+)

Syntax	*Example*

`string1 + string2`
 	
`variable1 + variable2`
	`first_name = "Aina"`
	`last_name = "Ali"`
	
	`first_name + last_name`

5 List (`list`)

- *Definition* : ***A list is a collection that can store multiple values in one variable.***

- Lists are written using square brackets [ ].

Examples
`marks = [80, 75, 90]` `names = ["Aina", "Ali", "Mei"]`

- *At this stage, you only need to recognize a list and its basic form.*
- *Lists will be discussed in detail in Topic 6.*

Data Types : Summary

A data type describes the kind of value stored in a variable.

Data Type	Python Name	Description
Integer	`(int)`	Whole numbers
Float	`(float)`	Numbers with decimal points
Boolean	`(bool)`	True or False values
String	`(str)`	Text or sequence of characters
List	`(list)`	A collection of multiple values



Review Activity : Data Type

Try this ! **Exercise 1**

Which of the following is a float value?

A) `"3.5"`
B) `3.5`
C) `3`
D) `'3'`



Review Activity : Data Type

Try this !  **Exercise 3**

What is the number of characters in `name`?

```
name = "Ali"
len(name)
```

A) 2
B) 3
C) 4
D) 5



Review Activity : Data Type

Try this ! **Exercise 4**

What is the result of the following string concatenation?

```
first_name = "Ali"
last_name = "Ahmad"

first_name + last_name
```

A. Ali Ahmad
B. AliAhmad
C. Ali+Ahmad
D. Ali Ahmad+

Table of Contents

Components of a Python program:

- ✅ identifiers
- ✅ variables
- ✅ reserved words/keywords
- ✅ data types
- output statements
- input statements
- import statements
- comments
- indentation

Topic 4: Python Programming

Learning Outcomes:

4.3 Identify components of a Python program:

- identifiers
- variables
- reserved words/keywords
- data types
- output statements
- input statements
- import statements
- comments
- indentation

COMPUTER PROGRAMMING

LECTURE NOTES

CS125

Table of Contents

Components of a Python program:

- ✅ identifiers
- ✅ variables
- ✅ reserved words/keywords
- ✅ data types
- ❑ output statements
- ❑ input statements
- ❑ import statements
- ❑ comments
- ❑ indentation

COMPUTER PROGRAMMING

LECTURE NOTES

CS125

Table of Contents

C. BASIC INPUT OUTPUT

5. Output statements
6. Input statements

COMPUTER PROGRAMMING

LECTURE NOTES

CP115

5. OUTPUT STATEMENT

Function of Output Statement:-
to display information on the screen.

Basic syntax	
`print(item)`	item can be text, a variable, or more than one item.

Remember: `print()` displays a result. It does not store a value.

Displaying Text Using `print()`

- To display text, place the text inside quotation marks.

Example	*Displayed result*
`print("Hello")`	`Hello`

Text can use double quotation marks:	*or single quotation marks:*
`print("Welcome to Python")`	`print('Welcome to Python')`

Displaying a Variable Value

- A variable name can be placed inside `print()` to display the value stored in the variable.

Example	*Displayed result*
`student_name = "Aina"` `print(student_name)`	`Aina`

Do not place quotation marks around the variable name.	
❌ `print("student_name")`	displays the text `student_name`, not the stored value.

Displaying Text and a Variable Value

- The `print()` function can display text and a variable value in the same statement.

Example	*Displayed result*
`student_name = "Aina"` `print("Student name:", student_name)`	`Student name: Aina`

`"Student name:"` → text

`student_name` → variable

A comma separates the items inside `print()`.

Displaying Multiple Items Using Commas

- Use commas to display two or more items in one `print()` statement.

Basic syntax
`print(item1, item2, item3)`



- *Python automatically inserts a space between items separated by commas.*

Example	***Displayed result***

`name = "Aina"`
`mark = 85`
`print("Name:", name, "Mark:", mark)`
	`Name: Aina Mark: 85`

Displaying the Result of `len()`

- `len()` function was introduced to find the number of characters in a string. (refer previous lecture notes 4.4 Basic String Operation)

```
len(variable_name)      → finds the number of characters
print(...)              → displays the returned value
```

Using `len()` with `print()`	***Displayed result***
`name = "Aina"` `print(len(name))`	`4`

`"Aina"` contains 4 characters.

Displaying the Result of String Concatenation

- The + operator joins two or more strings. The joined string can be displayed using `print()`. → (refer previous lecture notes 4.4 Basic String Operation)

```
string1+ " " + string2          → joins the strings
print(...)                      → displays the joined string
```

Using concatenation with `print()`	*Displayed result*

`first_name = "Aina"`
`last_name = "Ali"`
`print(first_name + " " + last_name)`
	`Aina Ali`

`print()` Function: Summary

Purpose	*Example*	*Displayed Result*
Display text	`print("Hello")`	`Hello`
Display variable	`print(name)`	`Aina`
Display text + variable	`print("Name:", name)`	`Name: Aina`
Display multiple items	`print(name, mark)`	`Aina 85`
Display number of characters	`print(len(name))`	`4`
Display concatenated strings	`print(first_name + " " + last_name)`	`Aina Ali`

Input Statement: `input()` Function

`input( )function`
• allows your Python program to **interact with the user** by accepting **keyboard input *to display text, numbers, or other data types on the screen.***
• to accept data entered by the user.

Input Statement: `input()` Function

Basic Syntax	*Example*
`input("prompt message")`	`input("Enter your name: ")`

- *When `input()` is executed, the program waits for the user to enter a value.*

Storing User Input in a Variable

Basic Syntax	*Example*
`variable_name = input(“prompt message”)`	`name = input("Enter your name: ")`

- The value entered by the user can be stored in a variable.

Storing User Input in a Variable

Basic Syntax	*Example*
`variable_name = input(“prompt message”)`	`name = input("Enter your name: ")`

- The value entered by the user can be stored in a variable.
- The user input is always returned as a **string** (data type: str).



Default Data Type Returned by `input()`

- The `input()` function always returns the user's input as a `string (str)`.

Example	*If the user enters*
`age = input("Enter age: ")`	`18`

Python stores the entered value as the string "18", not the integer 18.

Identifying Data Type Using `type()`

- The `type()` function is used to identify the data type of a value or variable

Syntax	*Example*
`type(object)`	`age = input("Enter age: ")` `type(age)`

- **`object`** → *The variable or value whose type you want to check.*
- ✅ Returns the **type/class** of the object.

Even if the user enters 18, `input()` returns a string.

Identifying Data Type Using `type()`

- The `type()` function can identify the data type of a variable or a value directly.

Using a Variable	*Using a Value Directly*
Basic form: `type(variable_name)`	Basic form: `type(value)`
To display: `print(type(variable_name))`	To display: `print(type(value))`

- `type()` identifies the data type
- `print()` displays the returned result

`type()` Function

Examples of Using `type()`

Using a Variable	*Using a Value Directly*

`num = 42`
`print(type(num))`
 	
`Result: <class 'int'>`
	`print(type(42))`
	`print(type(3.14))`
	`print(type("Hello"))`
	`print(type(True))`

- *The value can also be checked directly without assigning it to a variable first.*

`type()` Function

Examples of Using `type()`

```
num = 42
print(type(num))
```

```
<class 'int'>
```

Integer

```
pi = 3.14
print(type(pi))
```

```
<class 'float'>
```

Float

```
name = "Ali"
print(type(name))
```

```
<class 'str'>
```

String

```
is_ready = True
print(type(is_ready))
```

```
<class 'bool'>
```

Boolean

```
print(type(42))
print(type(3.14))
print(type("Hello"))
print(type(True))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

- determine the data type of an object

Type Casting

Convert the Input String to the Correct Data Type

Definition
The process of converting a value from one data type to another data type.

- Key idea: casting changes the data type of a value.

Basic syntax
`data_type(value)`

`data_type` → the required data type

`value` → the value to be converted

Type Casting

Type Casting with User Input

- The `input()` function returns a string. Casting is used when another data type is required.

Without casting	With casting
`age = input("Enter age: ")` *`age` is stored as a string.*	`age = int(input("Enter age: "))` *`age` is stored as an integer.*

- *`input()` accepts the data → `int()` converts the returned string to an integer.*

Type Casting

Common Type Casting Functions

Casting Function	Basic Syntax	Purpose
`int()`	`int(value)`	Convert a value to integer
`float()`	`float(value)`	Convert a value to float
`str()`	`str(value)`	Convert a value to string
`bool()`	`bool(value)`	Convert a value to Boolean

Type Casting

Examples of Type Casting

Conversion	Casting Expression	Result
String → Integer	`int("25")`	`"25" → 25`
String → Float	`float("3.5")`	`"3.5" → 3.5`
Integer → String	`str(100)`	`100 → "100"`
Integer → Boolean	`bool(1)`	`1 → True`

- The original value is converted into the data type specified by the casting function.

Common Mistakes

Mistake	Explanation
Forgetting to store the input	`input("Enter name: ")` accepts the input, but it is not stored for later use. Use `name = input("Enter name: ").`
Forgetting type casting	`input()` returns a string. For example, if the user enters `5`, it is stored as `"5"`. Use `int(input(...))` when an integer is required.
Using the wrong casting function	The casting function must match the required data type. Example: use `float(input(...))` for a decimal value such as `3.5`.
No prompt message	`input()` does not tell the user what information to enter. Use a clear prompt such `as input("Enter your name: ").`

Summary

Feature	Description
Function name	`input()`
Returns	Always returns `str`
Used for	Accepting keyboard input from user
Prompt	Optional message inside the function
Conversion / casting	Use `int()`, `float()` for numeric values

Summary

Purpose	Function / Example	Description
Accept input	`input()`	Returns a string
Store input	`name = input()`	Stores user input
Identify type	`type(name)`	Identifies data type
Convert to integer	`int(input())`	Returns integer
Convert to float	`float(input())`	Returns float
Other casting	`str(), bool()`	Convert when required

- Key idea: `input()` accepts data; type() identifies its type; casting converts its type

COMPUTER PROGRAMMING

LECTURE NOTES

CS125

Table of Contents

Components of a Python program:

- identifiers
- variables
- reserved words/keywords
- data types
- output statements
- input statements
- import statements
- comments
- indentation

Topic 4: Python Programming

Learning Outcomes:

4.3 Identify components of a Python program:

- identifiers
- variables
- reserved words/keywords
- data types
- output statements
- input statements
- import statements
- comments
- indentation

COMPUTER PROGRAMMING

LECTURE NOTES

CS125

Table of Contents

Components of a Python program:

- [x] identifiers
- [x] variables
- [x] reserved words/keywords
- [x] data types
- [x] output statements
- [x] input statements
- [ ] import statements
- [ ] comments
- [ ] indentation

Table of Contents

D. USING PYTHON MODULE

7. Import statement

LECTURE NOTES

7. IMPORT STATEMENT

Purpose of Import Statement:-
to bring in modules and their functionalities into the program.

- This allows you to use pre-defined functions and constants without having to define them yourself.

Importing the math Module

Syntax

```
import math
```

LECTURE NOTES

7. IMPORT STATEMENT

In the case of mathematical operations, Python provides the math module, which includes important constants and functions such as:

1. `math.pi` → *The mathematical constant π (pi).*
2. `math.pow(x, y)` → *Raises x to the power of y.*
3. `math.sqrt(x)` → *Computes the square root of x.*

1

math.pi

- `math.pi` is a mathematical constant representing the value of π (pi), approximately 3.141592653589793.

Import the Entire math Module

Syntax	*Example*

`import math`
 	
`print(math.pi)` # *Using pi*
	`import math` # *Import the entire math module*
	
	`radius = 7`
	
	# *Using math.pi to calculate circumference*
	`circumference = 2 * math.pi * radius`
	
	`print("Circumference of the circle:", circumference)`

2 **`math.pow(x,y)`**

- It always returns a **floating-point** number.

Import the Entire math Module

Syntax	***Example***

`import math`
 	
`print(math.pow(x,y))` *# Using pow( )*
	`import math` *# Import the entire math module*
	
	`base = 3`
	`exponent = 4`
	
	# Using math.pow() to calculate 3^4
	`result = math.pow(base, exponent)`
	
	# Using string concatenation (+)
	`print(base, "raised to the power of", exponent, "is:", result)`

Answer : 3 raised to the power of 4 is: 81.0

3 `math.sqrt(x)`

- The argument x must be **non-negative** (≥ 0).
- The result is always a **floating-point** number.

Import the Entire math Module

Syntax	*Example*

`import math`
 	
`print(math.sqrt(x))` *# Using sqrt( )*
	`import math` *# Import the entire math module*
	
	`number = 64`
	
	# Using math.sqrt() to find the square root of 64
	`square_root = math.sqrt(number)`
	
	# Displaying the result
	`print("The square root of", number, "is:", square_root)`

Answer : The square root of 64 is: 8.0

Table of Contents

E. PROGRAM FORMATTING

8. Comments
9. Indentation

COMPUTER PROGRAMMING

LECTURE NOTES

CS125

8. COMMENT

Definition of Comment:-
A line of text within the code that explain what the code is doing and is ignored by the interpreter during execution.

When writing comments:

- The # character should be followed by a single space. Ex: *# End of menu* is easier to read than *#End of menu*.
- Comments should explain the purpose of the code, not just repeat the code itself. Ex: *# Get the user's preferences* is more descriptive than *# Input item1 and item2*

COMPUTER PROGRAMMING

LECTURE NOTES

CS125

8. COMMENT

Purposes of Comment:-
1. **Code Documentation** – Helps explain the purpose of code for other developers. 2. **Debugging Assistance** – Comments can be used to temporarily disable lines of code for testing. 3. **Increasing Readability** – Well-commented code is easier to understand and maintain. 4. **Collaboration** – Makes it easier for multiple developers to work on the same project. 5. **Docstrings for APIs** – Helps generate documentation for libraries and frameworks.

COMPUTER PROGRAMMING

LECTURE NOTES

CS125

8. COMMENT

Comments can be written

1. before a line of code

```
# This is a comment
print("Hello, World!")
```

2. can be placed at the end of a line, and Python will ignore the rest of the line

```
print("Hello, World!") # This is a comment
```

LECTURE NOTES

Python has two main types of comments:

1 Single-line Comments

- Uses the **#** symbol.
- Everything after # on that line is ignored by the interpreter.
- Used for short explanations or notes.

```
# This is a single-line comment
x = 10 # Assigning value to x
print(x) # Printing x
```

Python has two main types of comments:

2 Multi-line (Block) Comments

- Uses triple quotes (''' or """) as multi-line strings, which is often called **docstrings**.

```
"""
This is a multi-line comment.
It is written using triple quotes.
This program is to print
Hello, World!
"""

print("Hello, World!")
```



Review Activity : Comment

Try this ! **Exercise 1**

The main purpose of writing comments is to _____.

a. avoid writing syntax errors

b. explain what the code does

c. make the code run faster



Review Activity : Comment

Try this !  **Exercise 2**
Which symbol is used for comments in Python?

LECTURE NOTES

Definition of Indentation:-
use of whitespace (spaces or tabs) at the beginning of a line to define the structure of the code.

- Unlike other programming languages (such as Java or C++) that use curly braces {} to define blocks of code, Python relies on indentation to determine the grouping of statements.

Importance of Indentation

1 Defines Code Blocks

- In Python, indentation is used to indicate blocks of code, such as loops, functions, and conditionals.

Example

```
if True:
    print("This is inside the if statement")
print("This is outside the if statement")
```

Importance of Indentation

2. **Ensures Readability**
   - Proper indentation makes code more readable and easier to understand.
3. **Maintains Consistency**
   - Python enforces a consistent indentation style, which helps developers write cleaner and more structured code.

Importance of Indentation

4 Avoids Syntax Errors

- Since indentation is mandatory in Python, incorrect indentation leads to IndentationError.

Example of incorrect indentation:

```
if True:
print("Hello")   # This will cause an IndentationError
```

Examples of Improper Indentation

1 ***Example of Indentation Error in an if Statement***

```
age = 18
if age >= 18:
print("You are allowed to vote")   # No indentation, causes IndentationError
```

No Indentation

Examples of Improper Indentation

2 ***Example of Indentation Error in a `for..` Loop***

```
for i in range(5):
print(i)                    # No indentation, causes IndentationError
```



Examples of Improper Indentation

3 ***Example of Indentation Error in a Function***

```
def greet():
print("Hello, World!")          # No indentation, causes IndentationError


greet()
```

No Indentation

Examples of Improper Indentation

4 ***Example of Mixing Spaces and Tabs (Bad Practice)***

```
def add(a, b):
    result = a + b   # Indented using spaces
  result += 1        # Indented using a tab (causes error)
    return result
```

Python Indentation Rules

1. Use **4 spaces per indentation level** (recommended by PEP 8).
2. Avoid mixing spaces and tabs to prevent errors.
3. Ensure all statements in a block have the same indentation.

COMPUTER PROGRAMMING

LECTURE NOTES

CS125

Table of Contents

Components of a Python program:

- identifiers
- variables
- reserved words/keywords
- data types
- output statements
- input statements
- import statements
- comments
- indentation

LECTURE NOTES

Topic 4: Python Programming

Learning Outcomes:

4.5 Identify the use of assignment, arithmetic operators, relational and logical operators . (1st hour)

What is an **Operator**?
A symbol or keyword used to perform an operation on values.
Operands
The values used by an operator

LECTURE NOTES


```
5 + 3

5 and 3 are operands
+ is the operator
```

- Operators can calculate, compare, assign, or combine conditions.

What is an **Expression**?

The combination of variables, literals, operators, and parentheses.

COMPUTER PROGRAMMING

LECTURE NOTES


```
total = 10 + 5 * 2
```

Term	Meaning	Example
Literal	A fixed value written directly in the program	`10, 5, 2`
Variable	A name used to store a value	`total`
Operator	A symbol that performs an operation	`=, +, *`
Parentheses	Symbols used to group part of an expression	`(5 * 2)`
Expression	A combination of values, variables, operators, and/or parentheses that produces a value	`10 + 5 * 2`

LECTURE NOTES


```
total = 10 + 5 * 2

Right side expression:
10 + 5 * 2

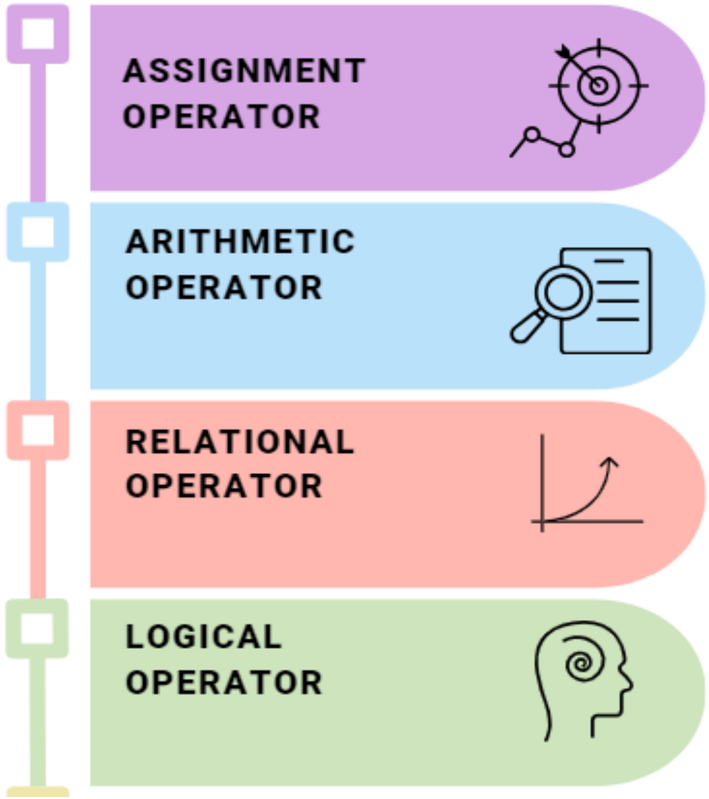
Result: 20
```

Python evaluates the expression first

- Python evaluates an expression to produce one result.
- The result can be a number, string, or Boolean value.

LECTURE NOTES

- Here's a list of different types of Python operators.



Assignment Operators (=)

- The assignment operator = stores a value in a variable.
- The variable is written on the left side.
- The value or expression is written on the right side.
- Read it as “is assigned the value”, not “equals”.

```
age = 18
name = "Mira"
price = 12.50
is_student = True
```

Key message

One equal sign (=) means assignment.

It stores data for later use.

- The simplest form of an assignment statement is the following:

```
variable_name = value
```

Any Python variable name, such as **`totalIncome`** or **`taxRate`**.

Any Python expression, such as **`" " * 10 + "Python"`**

LECTURE NOTES

ASSIGNMENT OPERATORS

Basic Assignment Syntax

```
variable_name = value
```

Example

```
mark = 85
```

- `mark` is the variable name.
- `85` is the value.
- `=` stores `85` into `mark`.
- The value can be used again later.

LECTURE NOTES

Example

```
total = 3 + 4 * 2
```

Step 1: Evaluate `4 * 2` → `8`.

Step 2: Evaluate `3 + 8` → `11`.

Step 3: Store `11` **in** `total`.

Right side first → then store

Assignment vs Comparison

Assignment: =	Comparison: ==
• Used to store a value. • Example: `floor = 13` • Meaning: store `13` in `floor`. • Used outside a condition test.	• Used to compare two values. • Example: `floor == 13` • Result: `True` or `False`. • Used inside a condition test.

Common mistake: using = when you mean ==

Assignment vs Comparison

```
x = 5    # This assigns 5 to x
x == 5   # This checks if x is equal to 5 (gives True or False)
```

Variables Can Be Reassigned

- Variables can be reassigned

```
x = 5
x = 8    # x now holds 8, not 5 anymore
print(x)  # Output: 8
```

How??

- A variable is created the first time it is assigned a value.
- Assigning a value to an existing variable replaces the previously stored value.

Variables Can Be Reassigned

Example

```
score = 50
score = 80
print("Final score:", score)
```

- `score = 50`
  The variable score is created and assigned the value 50.
- `score = 80`
  The same variable score is assigned a new value, 80.
  The previous value 50 is replaced.
- `print("Final score:", score)`
  Python displays the current value stored in score, which is 80.

Multiple Assignment

Same value to several variables	Different values to several variables

`a = b = c = 0`
`a` **stores** `0`
`b` **stores** `0`
`c` **stores** `0`
	`x, y, z = 1, 2, 3`
	`x` **stores** `1`
	`y` **stores** `2`
	`z` **stores** `3`

Multiple Assignment

- Examples

a. Assign the same value to multiple variables

```
a = b = c = 0
print(a, b, c)
```

Output :

```
0 0 0
```

b. Assign multiple values to multiples variables

```
x, y, z = 1, 2, 3
print(x, y, z)
```

Output :

```
1 2 3
```

LECTURE NOTES

ASSIGNMENT OPERATORS

- You can assign different types of data

```
x = 10                  # Integer
pi = 3.14               # Float / Double
name = "Mira"           # String
is_student = True       # Boolean
```

Common Assignment Mistakes

- Writing the value on the left side: `10 = x` ✗
- Using `==` for assignment: `x == 10` ✗
- Using a variable before assigning a value.
- Forgetting that `True` and `False` must start with capital letters.
- Using invalid variable names such as `2mark` or `final score`.

Correct example: `final_score = 90`

Arithmetic Operators

- An **arithmetic operators** are used to perform mathematical operations.

  Example:

  + , - , /, *, %, **, //

- An **arithmetic expression** consists of operands and operators combined in a manner

  Example:

```
result = 3 + 4 * 2
```

Arithmetic Operators

Operator	Meaning	Syntax
**	Exponentiation	a ** b
*	Multiplication	a * b
/	Division	a / b
//	Quotient or Floor division * positive value	a // b
%	Remainder or modulus	a % b
+	Addition	a + b
-	Subtraction	a - b

Arithmetic Operators

Operator	Example	Result
**	5 ** 2	7
*	5 * 2	3
/	5 / 2	10
//	5 // 2 (*positive value*)	2.5
%	5 % 2	1
+	5 + 2	25
-	5 - 2	2

Precedence Rules

Operation	Precedence	Associativity	Description
Exponentiation (**)	Highest	Right to Left	Evaluated first
Multiplication, Division, Remainder	Medium	Left to Right	
Addition, Subtraction	Low	Left to Right	Evaluated before addition and subtraction
Assignment (=, +=, etc.)	Lowest	Right to Left	
Parentheses ()	Overrides All	–	Evaluated after multiplication/division

COMPUTER PROGRAMMING

LECTURE NOTES

Precedence Rules

Operation	Example / Notes
Exponentiation (**)	2 ** 3 ** 2 → 2 ** (3 ** 2)
Multiplication, Division, Remainder	10 * 2 / 5 % 3
Addition, Subtraction	4 + 3 - 1
Assignment (=, +=, etc.)	x = y = 10 assigns 10 to both y and x
Parentheses ()	(3 + 2) * 4 forces addition first

- Operations with **equal precedence** are usually **left to right**.
- **Exceptions**: ** and = are **right to left**.
- Use **parentheses** to make evaluation order explicit.

Addition and Subtraction

```
a = 10
b = 3

print(a + b)    # 13
print(a - b)    # 7
```

- Identify variable values first.
- Replace a with 10 and b with 3.
- Evaluate 10 + 3.
- Evaluate 10 - 3.

Use simple numbers before using input()

Multiplication and Division

```
quantity = 4
price = 2.50

total = quantity * price
average = total / 2
```

`quantity * price → 4 * 2.50 → 10.0`

`total / 2 → 10.0 / 2 → 5.0`

Division / usually produces a float result.

How can we display the average value?

Division / vs Floor Division //

Division /	Floor division //
• Produces quotient with decimal. • Example: `7 / 2` • Result: `3.5`	• Produces whole-number quotient for positive values. • Example: `7 // 2` • Result: `3`

For this topic, focus on positive values only

Remainder / Modulus %

```
7 % 2
```

```
7 % 2 → 1
```

- 7 divided by 2 gives quotient 3.
- `2 × 3 = 6`.
- `7 - 6 = 1`.
- So, `7 % 2` gives remainder 1.

Useful later for even/odd checking

Exponentiation **

```
2 ** 3
```

```
2 ** 3 → 8
```

- 2 ** 3 means 2 to the power of 3.
- 2 × 2 × 2 = 8.
- Do not use ^ for power in Python.

Common mistake: 2 ^ 3 is not exponentiation in Python

Examples of arithmetic expressions and their values

Expression	Evaluation	Value
`5 + 3 * 2`	`5 + 6`	`11`
`(5 + 3) * 2`	`8 * 2`	`16`
`6 % 2`	`0`	`0`
`2 * 3 ** 2`	`2 * 9`	`18`
`-3 ** 2`	`-(3 ** 2)`	`-9`
`(3) ** 2`	`9`	`9`
`2 ** 3 ** 2`	`2 ** 9`	`512`
`(2 ** 3) ** 2`	`8 ** 2`	`64`
`45 / 0`	`Error: cannot divide by 0`	
`45 % 0`	`Error: cannot divide by 0`	

- Mixing integers and floating-point values in an arithmetic expression yields a floating-point value.

Example:

If an expression contains both int and float, **the result will always be a float.**

```
a = 5          # int
b = 2.0        # float

print(a + b)         # 7.0 (int + float → float)
print(a * b)         # 10.0
print(a / b)         # 2.5
print(a // b)        # 2.0 (floor division but result is float)
```

Evaluate Arithmetic Expression Step by Step

```
result = 3 + 4 * 2
```

```
result → 11
```

Step 1: Multiplication first → 4 * 2 = 8

Step 2: Addition next → 3 + 8 = 11

Step 3: Assignment → result stores 11

Which part is calculated first?

Mixing int and float

```
a = 5          # int
b = 2.0        # float

print(a + b)    # 7.0
print(a * b)    # 10.0
print(a // b)   # 2.0
```

- If an arithmetic expression contains int and float, the result is usually float.
- This helps students understand why 7.0 appears instead of 7.
- Keep the explanation simple.

String and Arithmetic Operators

```
"Hi" + "Mira"   → "HiMira"
"Hi" * 3        → "HiHiHi"
"Hi" - "Mira"   → error
```

- Strings do not support normal arithmetic calculations.
- + with strings means concatenation, not addition.
- * with a string repeats the string.
- Use this as a reminder, because string concatenation was introduced earlier.

Do not treat string + as arithmetic addition

COMPUTER PROGRAMMING

LECTURE NOTES

CS125

ARITHMETIC OPERATORS

Example 1

```
val1 = 2
val2 = 3

# using the addition operator
total = val1 + val2
print(total)
print("Total = ", total)
```

Output :

```
5
Total =  5
```

LECTURE NOTES

Example 2

```
a = 21.0
b = 10
# Addition
print ("a + b : ", a + b)
# Subtraction
print ("a - b : ", a - b)
# Multiplication
print ("a * b : ", a * b)
# Division
print ("a / b : ", a / b)
# Modulus
print ("a % b : ", a % b)
# Exponent
print ("a ** b : ", a ** b)
# Floor Division
print ("a // b : ", a // b)
```

Output :

```
a + b :  31.0
a - b :  11.0
a * b :  210.0
a / b :  2.1
a % b :  1.0
a ** b :  16679880978201.0
a // b :  2.0
```

Example 3 : Multiplication

```
length = 7

width = 3

area = length * width

print(area)   # Output: 21
```

LECTURE NOTES

TRY THIS !

```
print (((5 + 4) / 3) * 2)
print((2 ** 4), (2 * 4.), (2 * 4))
print((-2 / 4), (2 / 4), (2 // 4), (-2 // 4))
print((2 % -4), (2 % 4), (2 ** 3 ** 2))
```

Output : ????

TRY THIS !

What is the expected output of the following snippet?

(a)

```
a = '1'
b = "1"
print(a + b)
```

(b)

```
print(9 % 6 % 2)

print((2 ** 4), (2 * 4.), (2 * 4))

print((-2 / 4), (2 / 4), (2 // 4), (-2 // 4))

print((2 % -4), (2 % 4), (2 ** 3 ** 2))
```

LECTURE NOTES

ANSWER TRY THIS !

What is the expected output of the following snippet?

(a)

```
a = '1'
b = "1"
print(a + b)
```

Output :

```
11
```

(b)

```
print(9 % 6 % 2)

print((2 ** 4), (2 * 4.), (2 * 4))

print((-2 / 4), (2 / 4), (2 // 4), (-2 // 4))

print((2 % -4), (2 % 4), (2 ** 3 ** 2))
```

```
1
16 8.0 8
-0.5 0.5 0 -1
-2 2 512
```

LECTURE NOTES

Topic 4: Python Programming

Learning Outcomes:

4.5 Identify the use of assignment, arithmetic operators, relational and logical operators . (2nd hour)

Equ
al to

COMPUTER PROGRAMMING

LECTURE NOTES

What Are Relational Operators?

- Relational operators are commonly used to form a condition in `if` statement for selection, and `while` or `for` statements for looping.
- Relational operators compares two values and return a Boolean result (**`True`** or **`False`**) .

Description	Relational Operators	Example
Equal to	==	5 == 5 → True
Not equal to	!=	8 != 8 → False
Less than	<	3 > 7 → False
Greater than	>	3 < 7 → True
Less than or equal to	<=	5 >= 5 → True
Greater than or equal to	>=	4 <= 6 → True

Relational Operator VS Assignment Operator

Relational operator
==
To compare two values to determine whether they are equal

VS

Assignment operator
=
To assign a value to a variable

Example:
```
if floor == 13:
# Test whether floor equals 13
```

Example:
```
floor = 13
# Assign 13 to floor
```

- You must remember to use **==** inside tests and to use **=** outside tests.

Relational Operator Examples

Expression	Value	Comment
`3 <= 4`	`True`	3 is less than 4; <= tests for "less than or equal".
`3 =< 4`	**Error**	The "less than or equal" operator is <=, not =<. The "less than" symbol comes first.
`3 > 4`	`False`	> is the opposite of <=.
`4 < 4`	`False`	The left-hand side must be strictly smaller than the right-hand side.
`4 <= 4`	`True`	Both sides are equal; <= tests for "less than or equal".
`3 == 5 - 2`	`True`	== tests for equality.
`3 != 5 - 1`	`True`	!= tests for inequality. It is true that 3 is not 5 – 1.
`3 = 6 / 2`	**Error**	Use == to test for equality.
`1.0 / 3.0 == 0.333333333`	`False`	Although the values are very close to one another, they are not exactly equal.
`"10" > 5`	**Error**	You cannot compare a string to a number.

What Are Logical Operators?

- Logical operators sometimes are referred to as Boolean Operators.
- Logical operators combine multiple conditions and return a Boolean result (`True` or `False`).

Description	Logical Operators	Example
True if both are true	and	(5 > 3) and (3 < 7) → True
True if at least one is true	or	(5 > 3) or (3 > 7) → True
Inverts the Boolean value	not	not (5 > 3) → False

Precedence of Operators

HIGHEST



LOWEST

Precedence Level	Operator	Explanation
1 (highest)	( )	Parentheses
2	**	Exponentiation
3	-a, +a	Negative, positive argument
4	* , / , // , %, @	Multiplication, division, floor division, modulus, at
5	+, -	Addition, subtraction
6	< , <=, >, >=, ==, !=	Less than, less than or equal, greater, greater or equal, equal, not equal
7	not	Boolean Not
8	and	Boolean And
9	or	Boolean Or

Example 1: The relational operator (==) usage.

== operator checks if two values are **equal**.

Write a program to determine if two input numbers are equal. If both numbers are the same, print “The numbers are equal”. Otherwise, print “The numbers are not equal”.

Syntax:

```
# Prompt the user to input two numbers
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

# Check if the numbers are equal
if num1 == num2:
    print("The two numbers are equal.")
else:
    print("The two numbers are not equal.")
```

Example 2: The relational operator (>, <) usage.

> operator checks if one value is **greater than** another.
< operator checks if one value is **less than** another.

Write a program that allows user to input a number. The program should determine whether the number is positive, negative, or zero, and print the appropriate message.

Syntax:

```
# Prompt the user to input a number
num = float(input("Enter a number: "))

# Determine if the number is positive, negative, or zero
if num > 0:
    print("The number is positive.")
elif num < 0:
    print("The number is negative.")
else:
    print("The number is zero.")
```

Example 3: The relational operator (! =) usage.

!= operator checks if two values are **not equal**.

Write a program that asks user to input their age. The program should check if the age is not equal to 18 and print an appropriate message.

Syntax:

```
# Prompt the user to input their age
age = int(input("Enter your age: "))

# Check if the age is not equal to 18
if age != 18:
    print("You are not 18 years old.")
```

Example 4: The logical operator (`not`) usage.

not operator inverts the Boolean value

Write a program that asks user to input a string. The program should print True if the string is not empty, and False if the string is empty. Use the `not` operator in your solution.

Syntax:

```
# Prompt the user to input a string
user_input = input("Enter a string: ")

# Check if the string is not empty using the 'not' operator
if not user_input:
    print("False")   # Empty string
else:
    print("True")    # Non-empty string
```

Explanation:

- The **not** operator is used to check if the string is empty. In Python, an empty string is considered **False**, and any non-empty string is considered **True**.
- If `user_input` is empty, **not** `user_input` will evaluate to **True**, and the program will print **False** (indicating the string is empty).
- If `user_input` is not empty, **not** `user_input` will evaluate to **False**, and the program will print **True** (indicating the string is not empty).

Example 5: The logical operator (`not`) usage.

not operator inverts the Boolean value

Write a program to determine if a number is not equal to 10. If it is not equal to 10, print "The number is not 10". Otherwise, print "The number is 10". Use the `not` operator in your solution.

Syntax:

```
# Input: Get a number from the user
number = int(input("Enter a number: "))

# Check if the number is not equal to 10 using 'not'
if not number == 10:
    print("The number is not 10")
else:
    print("The number is 10")
```

Explanation:

Condition with `not`

- The expression `not number == 10` inverts the condition.
- If `number == 10` is **`True`**, **`not`** makes it **`False`**.
- If `number == 10` is **`False`**, **`not`** makes it **`True`**.

COMPUTER PROGRAMMING

LECTURE NOTES

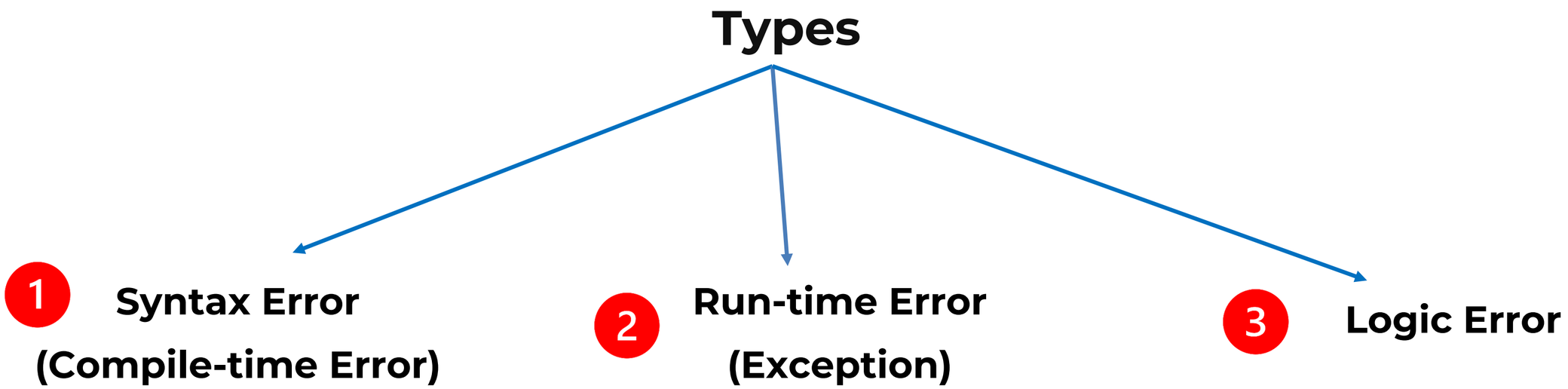
Topic 4: Python Programming

Learning Outcomes:

4.7 Explain types of programming errors: syntax, run-time and logic.

LECTURE NOTES

Definition of Programming Errors:-
Any mistake in a program's code that prevents it from running correctly or producing the correct output.



1 Syntax Error

Definition of Syntax Error:-

A violation of the programming language rules, detected when the code is translated into executable form (before the program runs).

LECTURE NOTES

1 Syntax Error

When Detected:	Before the program runs, during compilation/ interpretation • when code breaks the grammar rules of Python.
Example:	`print("Hello, World)` # *Missing closing quotation mark* Output: `SyntaxError: EOL while scanning string literal`
Fix	Correct the syntax according to Python rules.

1 Syntax Error

More examples ...

Example 1	*Missing parentheses in print*	`print "Hello, World!"`
	Error Message:	`SyntaxError: Missing parentheses in call to 'print'`
Example 2	*Using reserved keyword as a variable name*	`class = 10`
	Error Message:	`SyntaxError: invalid syntax`

2 **Run-time Error**

Definition of Run-time Error (Exception):-
An error that occurs while the program is executing, even if the syntax is correct.

2

Run-time Error

When Detected:	Only when the program executes. • Program starts but crashes due to an invalid operation.
Example:	`print(1 / 0)` *# Division by zero* Output: `ZeroDivisionError: int division or modulo by zero`
Fix	Handle the error using proper logic (e.g., avoid dividing by zero, use exception handling).

2 Run-time Error

More examples ...

Example 1	*using undefined variable – case-sensitive issue*	`num1 = 10`
		`print(Num1)  # Wrong case`
		(Python is case-sensitive)

	Error Message: • Program starts fine but crashes at run time when trying to use Num1 instead of num1.	`NameError: name 'Num1' is not defined`
Example 2	*Wrong type operation*	`age = "20"` `print(age + 5)`
	Error Message:	`TypeError: can only concatenate str (not "int") to str`

3 Logic Error

Definition of Logic Error:-
An error occurs when the program runs without crashing but produces incorrect results.
Program runs successfully but produces incorrect results due to flawed logic.

- *This usually happens due to mistakes in the program's logic (e.g., using + instead of * in a calculation).*

3 Logic Error

When Detected:	Only when analyzing program output. • *Program runs successfully but produces the wrong result because of flawed logic.*
Example:	`print("Hello, Word!")` *# Wrong spelling in the string* Output: `Hello, Word! (wrong output, but no syntax or run-time error)`
Fix	Debug and correct the program logic to match the intended result.

3 Logic Error

More examples ...

Example 1	*Wrong formula for average*	`marks = [80, 70, 90]`
		`average = sum(marks)` *# Forgot to divide by number of items*
		`print("Average =", average)`

	Output:	`Average = 240` *# Wrong (should be 80.0)*

3 Logic Error

More examples ...

Example 2	*Wrong condition in decision-making*	`age = 16`
		`if age > 16:`
		`    print("You are allowed to drive")`
		`else:`
		`    print("You are not allowed to drive")`

	Output:	`You are not allowed to drive` `# Wrong, because 16 should also be allowed`

Summary

Types of Programming Errors	Common Examples
Syntax Error	• Missing () in print • Using reserved words as variables (`class = 10`). • Missing colon after if, for, or while. • Unmatched brackets/parentheses `{ [ ) }`. • Missing or extra quotation marks `("Hello)`.
Run-time Error	• Index out of range (`numbers[5]` when list has 3 items). • Type mismatch `("10" + 5)`. • Division by zero `(10 / 0)`. • Case-sensitive mismatch (`num1` vs `Num1`). • File not found `(open("data.txt")` when file doesn't exist).
Logic Error	• Wrong operator (+ instead of *). • Wrong condition (`if age > 16` instead of `>= 16`). • Incorrect formula (`average = sum(marks)` without dividing). • Misplaced loop range (`for i in range(1, 5)` prints `1-4`, not `1-5`). • Printing wrong variable (printing `total` instead of `average`).